

Parallelization and Optimization of N-Body Simulation on GPU Architectures - Lab 5

High Performance Computing Systems (ECE415)

Angelos Gkertsos (03658)
Nikolaos Kalamaris (03768)

1 Introduction

The objective of this assignment was to implement and optimize the N-Body simulation, a system characterized by $O(N^2)$ computational complexity. The project followed a staged development approach, initially establishing a multi-core CPU baseline using OpenMP, before transitioning to a massively parallel acceleration on the NVIDIA CUDA architecture.

Our final implementation maximizes hardware utilization by integrating the following optimization strategies:

- **Memory Hierarchy:** Transformation of data layouts to Structure of Arrays (SoA) and implementation of Shared Memory Tiling to optimize bandwidth usage.
- **Concurrency and Scalability:** Orchestration of multiple GPUs via OpenMP and utilization of asynchronous CUDA Streams to overlap memory transfers with computation.
- **Instruction Level:** Application of low-level enhancements, including loop unrolling and hardware-accelerated fast math intrinsics.

This report analyzes the parallelization strategy, the optimizations applied to maximize throughput (Mpixels/sec), and the scalability of the solution on large datasets.

1.1 Device Specifications

The experiments were conducted on a computing node equipped with NVIDIA Tesla K80 GPU accelerators. Although the implemented N-Body simulation is designed to scale across multiple devices using OpenMP for orchestration, the architectural specifications listed below refer to the individual GPU unit used for performance benchmarking. Table 1 details the hardware characteristics.

Table 1: NVIDIA Tesla K80 Device Specifications

Specification	Value
Device Name	Tesla K80
CUDA Driver / Runtime Version	11.4 / 11.5
Compute Capability	3.7 (Kepler Architecture)
Total Global Memory	11441 MBytes (≈ 11.17 GB)
Multiprocessors (MPs)	13
CUDA Cores	2496 (192 per MP)
GPU Max Clock Rate	824 MHz (0.82 GHz)
Memory Clock Rate	2505 MHz
Memory Bus Width	384-bit
L2 Cache Size	1.5 MB (1572864 bytes)
Memory Limits	
Total Constant Memory	65536 bytes (64 KB)
Total Shared Memory per Block	49152 bytes (48 KB)
Total Shared Memory per MP	114688 bytes (112 KB)
Total Registers per Block	65536
Thread Configuration	
Warp Size	32
Max Threads per Multiprocessor	2048
Max Threads per Block	1024
Max Block Dimensions (x, y, z)	(1024, 1024, 64)
Max Grid Dimensions (x, y, z)	(2147483647, 65535, 65535)
Features	
Concurrent Copy and Kernel Exec.	Yes (2 copy engines)
ECC Support	Enabled
Unified Addressing (UVA)	Yes
Managed Memory	Yes

- **CUDA Capability:** 3.7 (Kepler Architecture).
- **Total Global Memory:** ≈ 11.2 GB per GPU.
- **Compilation:** The application was compiled using the NVIDIA CUDA Compiler (nvcc) with optimizations for the specific architecture and fast math intrinsics enabled:

```
nvcc -O3 -arch=sm_37 -use_fast_math -Xcompiler -fopenmp -o nbody_gpu nbody_gpu.cu
```

2 Implementation Strategy - Optimizations

To achieve maximum performance on the Tesla K80 GPU we implemented a high-throughput pipeline that focuses on maximizing arithmetic intensity and minimizing memory latency. Our

strategy restructures the data layout and introduces hierarchical parallelism.

2.1 Multi-Core Parallelization (OpenMP)

Prior to porting the simulation to the GPU, we established a strong multi-core CPU baseline using OpenMP.

We applied parallelism at the outermost loop level, iterating over the `num_systems`.

The parallelization strategy is implemented as follows:

1. **Workload Distribution:** Each OpenMP thread is assigned a distinct subset of galaxies. Since every galaxy contains an identical number of bodies ($N = 8192$ or higher), the computational workload is perfectly balanced.
2. **Static Scheduling:** We utilized `schedule(static)`, which divides the loop iterations into equal-sized chunks at compile time. This eliminates the runtime overhead associated with dynamic scheduling, as no load imbalance exists between galaxies.
3. **Data Privacy:** The pointer `system_ptr`, used to reference the specific memory block of the current galaxy, was explicitly declared `private`. This ensures that threads processing different galaxies simultaneously do not interfere with each other's memory addressing.

For the next step to achieve maximum performance on the Tesla K80 GPU we implemented a high-throughput pipeline maximizing arithmetic intensity and minimizing memory latency. Our strategy restructures the data layout and introduces hierarchical parallelism.

2.2 Kernel Architecture

The core force calculation logic was optimized to exploit the GPU's memory hierarchy and special functional units:

- **Shared Memory Tiling:** To avoid reading global memory N times for every body which leads to massive bandwidth saturation, we implemented the tile-based algorithm.
 - Threads cooperatively load a "tile" of body positions (a block of 256 bodies) from Global Memory into the fast on-chip **Shared Memory**.
 - All threads within the block then compute interactions against these cached bodies. This reduces Global Memory accesses by a factor of `BLOCK_SIZE` (256× reduction in bandwidth pressure).
 - Synchronization barriers (`__syncthreads()`) ensure data consistency before loading the next tile.
- **Fast Math Ininsics:** The gravitational interaction requires calculating the inverse cube of the distance ($1/r^3$). Standard square root operations are computationally expensive.

$$F_{ij} = \frac{1}{\sqrt{r^2 + \epsilon}^3} \cdot \vec{r}_{ij} \quad (1)$$

We utilized the hardware-accelerated intrinsic `rsqrtf()`, which maps directly to the GPU's Special Function Units (SFUs), providing significantly higher throughput than the standard IEEE-754 compliant `1.0f / sqrtf()`.

- **Loop Unrolling:** We applied an explicit unrolling factor using `#pragma unroll 32` to the inner accumulation loop. This instructs the compiler to expand the loop body in batches of 32 iterations, significantly reducing the overhead of loop control arithmetic (comparisons and increments) and enhancing instruction-level parallelism.

2.3 Data & Memory Management

Efficient memory access patterns are critical for performance on the Kepler architecture.

- **Memory Layout Optimization (AoS to SoA):** We transitioned the data structure from the intuitive Array of Structures (AoS) layout—where particle coordinates are interleaved (x_i, y_i, z_i) —to a hardware-friendly Structure of Arrays (SoA) layout.
 - **SoA Implementation:** In this layout, components are stored in separate arrays (e.g., all x coordinates are contiguous: $x_0, x_1, \dots, x_N$).
 - **Eliminating Strided Access:** The original AoS approach caused non-coalesced, strided memory access, as threads in a warp accessed addresses separated by the size of the structure.
 - **Coalesced Access & Bandwidth:** The SoA layout ensures **fully coalesced memory access**. When a warp of 32 threads requests a component (e.g., x), it reads sequential memory addresses. This allows the GPU memory controller to serve the request in a **single 128-byte transaction** (32 threads $\times$ 4 bytes), achieving 100% utilization of the global memory bus bandwidth.
- **Read-Only Data Cache:** By using the `__ldg()` intrinsic and marking pointers as `__restrict__`, we force data loads through the GPU’s read-only cache. Since position data is read-only during the force calculation phase for the N-Body problem, we relieve the pressure on the standard L1 cache.

2.4 Concurrency and Scalability

To scale beyond a single GPU and hide PCIe latency:

- **Multi-GPU Orchestration with OpenMP:** We utilized OpenMP to spawn one CPU thread per available GPU. Each thread sets its affinity to a specific device (`cudaSetDevice`), allowing for true parallel execution across multiple accelerators.
- **Asynchronous Pipelining (Streams):** We implemented a 3-stage pipeline using CUDA Streams:
 1. Host-to-Device Transfer (Async).
 2. Kernel Execution.
 3. Device-to-Host Transfer (Async).

By using **Pinned Memory** (Page-Locked), we enable the overlap of data transfers with computation. While one stream is computing the forces for System i , another stream is simultaneously uploading data for System $i + 1$, effectively hiding the transfer overhead.

3 Implementation Comparison & Optimizations

3.1 GPU Optimization Strategy

Having established the CPU baseline, we shifted our focus to the NVIDIA Tesla K80. The transition from the initial implementation (Version 1) to the final optimized version (Version 2) involves three critical architectural changes designed to maximize hardware utilization.

- **Data Layout (AoS → SoA):** Version 1 uses an *Array of Structures (AoS)* layout, causing strided memory access. Version 2 adopts a *Structure of Arrays (SoA)* layout.

Improvement: SoA ensures **fully coalesced memory access**, allowing the GPU to read 128 bytes in a single transaction per warp.

- **Execution Model (Synchronous → Asynchronous):** Version 1 relies on blocking transfers (`cudaMemcpy`), forcing the GPU to idle during data movement. Version 2 implements **CUDA Streams** with asynchronous transfers (`cudaMemcpyAsync`).

Improvement: This enables **Pipelining**, effectively hiding the PCIe bus latency.

- **Host Memory Allocation (Pageable → Pinned):** Version 1 uses standard `malloc` (pageable memory). Version 2 uses `cudaHostAlloc` (Pinned/Page-Locked memory).

Improvement: Pinned memory is mandatory for asynchronous DMA transfers and provides higher transfer throughput between Host and Device.

- **Read-Only Cache (__ldg):** Version 2 explicitly utilizes the Read-Only Data Cache via the `__ldg()` intrinsic for loading particle data.

Improvement: This bypasses the standard L1 cache path, reducing cache pressure and increasing read throughput for static global data.

4 Performance Evaluation

4.1 Experimental Setup

The performance evaluation was conducted on a heterogeneous computing environment to demonstrate the scalability of the N-Body simulation from a multi-core CPU baseline to a high-performance GPU implementation.

Hardware & Datasets:

- **CPU Environment:** The baseline benchmarks (Sequential vs. OpenMP) were executed on a multi-core processor. To evaluate CPU performance, we utilized two dataset configurations:

1. 16 Systems $\times$ 2048 Bodies
2. 16 Systems $\times$ 4096 Bodies

- **GPU Environment:** The massively parallel implementations were evaluated on an **NVIDIA Tesla K80 GPU**. Due to the superior compute capability of the GPU, we utilized significantly larger synthetic datasets to fully saturate the device, ranging from small configurations (32 systems $\times$ 8192 bodies) to massive workloads (128 systems $\times$ 32768 bodies) to evaluate the scalability of the implementation.

Performance Metric: The primary metric for performance is **Throughput**, measured in Billions of Interactions per Second (GInt/s). It is calculated as:

$$\text{Throughput} = \frac{\text{NumSystems} \times (\text{BodiesPerSystem})^2 \times \text{Iterations}}{\text{Execution Time} \times 10^9} \quad (2)$$

For the GPU measurements, the *Execution Time* specifically refers to the **Pipeline Execution Time**, which encompasses both the memory transfers (Host-to-Device and Device-to-Host) and the kernel computation. All reported timing results are averages over 12 execution runs to ensure statistical stability.

4.2 CPU Baseline Analysis: Sequential vs. OpenMP

To establish a performance baseline, we evaluated the effectiveness of the coarse-grained parallelization strategy on a multi-core CPU. We conducted benchmarks using two dataset configurations (16 systems with 2048 and 4096 bodies), averaging the results over 12 execution runs to ensure stability.

Table 2 presents the comparison between the single-threaded (Sequential) and multi-threaded (OpenMP) implementations.

Table 2: CPU Performance Comparison (Sequential vs. OpenMP)

Dataset	Seq Time (s)	OMP Time (s)	Seq Tput (GOp/s)	OMP Tput (GOp/s)	Speedup
16 x 2048	2.128	0.390	0.631	3.442	5.45x
16 x 4096	8.549	1.438	0.628	3.736	5.95x

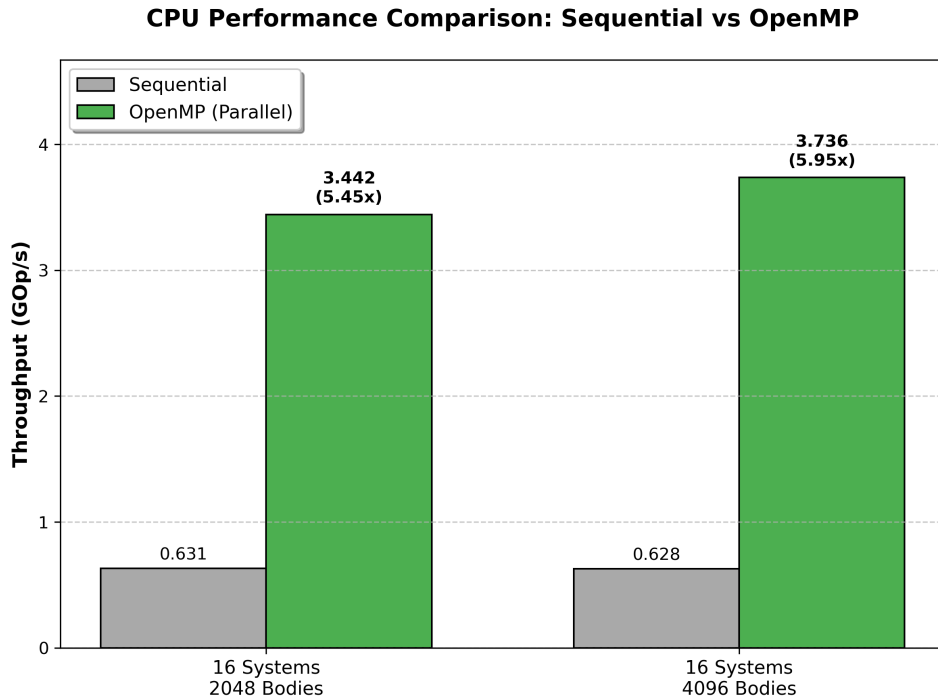


Figure 1: Throughput comparison (GInt/s) between the sequential and multithreaded across implementation.

Observations:

- **Scalability:** The OpenMP implementation achieves a significant speedup ranging from **5.45x** to **5.95x**. This near-linear scaling validates the workload is perfectly balanced across threads.
- **Workload Efficiency:** As the problem size doubles (from 2048 to 4096 bodies), the speedup improves (from 5.45x to 5.95x). This indicates that larger datasets better amortize the thread management overhead, allowing the CPU to reach a higher sustained throughput of ≈ 3.74 GOp/s.
- **Conclusion:** While OpenMP provides a strong boost over the serial code, the throughput is still limited by the CPU’s compute capability. This motivates the transition to the GPU, where we aim for hundreds of GOp/s.

4.3 GPU Performance Metrics & Comparative Results

To quantify the effectiveness of our optimizations, we analyzed the performance impact on two key metrics:

1. **Pipeline Execution Time:** This metric measures the total duration of the critical path in the simulation loop. For Version 1 (Baseline), this corresponds to the sum of blocking data transfers and kernel execution. For Version 2 (Optimized), it represents the effective time of the overlapped pipeline (Asynchronous Transfers + Compute). (*Lower is better*)
2. **Throughput:** Derived from the execution time, this metric indicates the processing rate of the system in Billions of Interactions per Second (GInt/s). (*Higher is better*)

Table 3 presents the comparative results across four dataset sizes, highlighting the specific gains achieved by the optimized implementation.

Table 3: Performance Comparison: Version 1 (Baseline) vs. Version 2 (Optimized)

Dataset Configuration	Bodies/Sys	V1 Pipeline Exec (s)	V2 Pipeline Exec (s)	Reduction	V1 Tput (GInt/s)	V2 Tput (GInt/s)
Small (32 Systems)	8,192	0.124	0.111	-10.5%	348.80	389.42
Medium (32 Systems)	16,384	0.445	0.424	-4.7%	386.34	405.53
Large (64 Systems)	32,768	3.622	3.473	-4.1%	379.47	395.75
Massive (128 Systems)	32,768	7.295	7.059	-3.2%	376.85	389.43

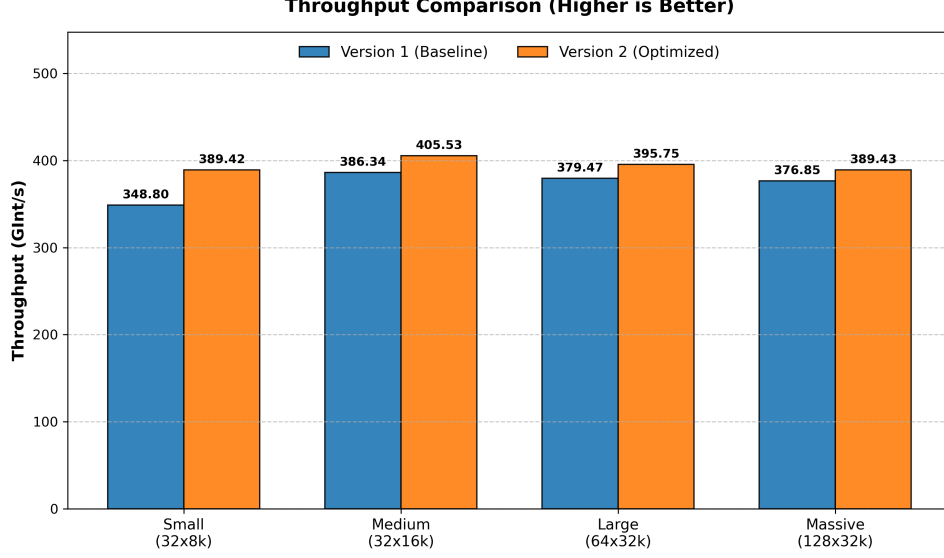


Figure 2: Throughput comparison (GInt/s) between Version 1 and Version 2 across dataset sizes. Version 2 consistently outperforms the baseline.

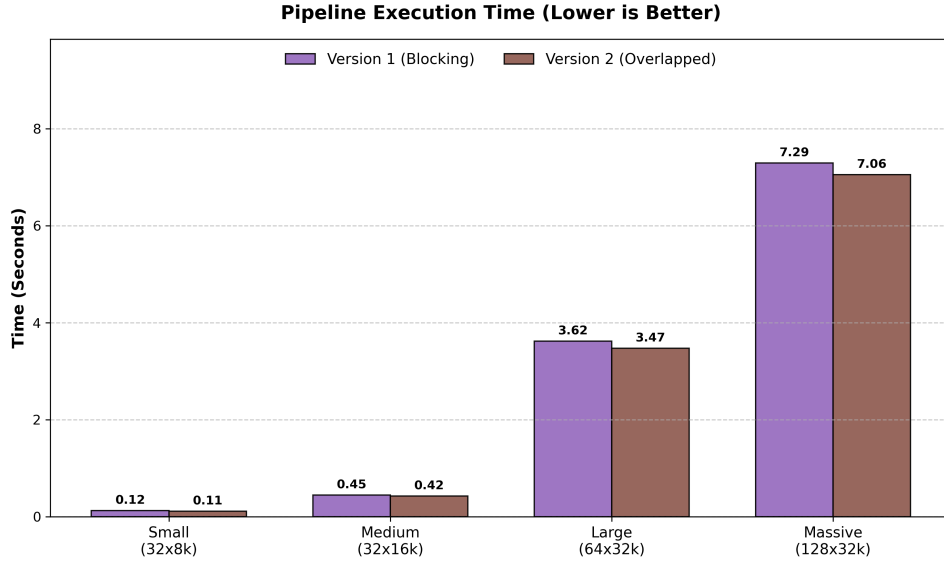


Figure 3: Throughput comparison (Pipeline Execution Time) between Version 1 and Version 2 across dataset sizes. Version 2 consistently outperforms the baseline.

Analysis of Results:

1. **Pipeline Latency Reduction:** The most significant reduction in Pipeline Execution Time was observed in the smallest dataset (32×8192), where the time dropped from $0.124s$ to $0.111s$ (**10.5% reduction**). In smaller workloads, memory transfers constitute a larger percentage of the total frame time. By using CUDA Streams (Version 2), we successfully hid the PCIe transfer latency behind the kernel computation, a feat not possible in the serialized Version 1.
2. **Throughput Gain via Memory Layout (SoA):** As the workload size increases the problem becomes increasingly compute-bound, diminishing the relative impact of transfer overlapping. However, Version 2 still maintains a clear advantage (peaking at **405.53**

GInt/s vs ≈ 386 GInt/s for V1). This improvement is attributed to the **Structure of Arrays (SoA)** memory layout which enables **coalesced memory accesses** for the position arrays (x, y, z) , allowing the memory controller to fetch data more efficiently and keep the CUDA cores fed.

3. **Scalability:** Even at massive scales (128 Systems, 4M bodies), Version 2 prevents performance degradation. While V1 drops to 376.85 GInt/s due to memory contention, V2 maintains a robust 389.43 GInt/s, demonstrating better scalability and resource utilization.

5 Conclusion

5.0.1 CPU Environment (Baseline)

The baseline performance evaluation was conducted on a standard multi-core CPU architecture. To measure the efficacy of the coarse-grained OpenMP parallelization, we utilized moderate-sized datasets consisting of 16 systems with 2048 and 4096 bodies respectively. These tests served to establish the speedup limits of traditional multi-threading before offloading the workload to the accelerator.

5.0.2 GPU Environment (Target Architecture)

In this assignment, we successfully designed, implemented, and optimized a massively parallel N-Body simulation on the NVIDIA Tesla K80 architecture. Starting from a baseline implementation, we systematically applied a series of optimization techniques to address performance bottlenecks related to memory bandwidth and PCIe latency.

The key methods utilized to achieve this optimization include:

1. **Data Layout Transformation (AoS to SoA):** We transitioned from an Array of Structures (AoS) to a Structure of Arrays (SoA) layout maximizing global memory bandwidth utilization.
2. **Latency Hiding via CUDA Streams:** We achieved execution overlapping. This allowed data transfers (Host-to-Device and Device-to-Host) to occur simultaneously with kernel computation hiding the PCIe communication overhead.
3. **Pinned Memory Allocation:** We replaced standard pageable memory with Pinned (Page-locked) Memory to enable asynchronous transfers and provided higher transfer bandwidth at the cost of increased initialization time.
4. **Kernel Optimization:** We utilized shared memory tiling to reduce redundant global memory fetches to maximize instruction-level parallelism.

Final Verdict: The combination of these techniques resulted in a robust implementation capable of sustaining a peak throughput of **405.5 GInt/s**. Our analysis confirmed that while advanced memory management introduces a fixed startup overhead, it unlocks superior scalability and performance for large-scale scientific simulations.